

บทที่ 2

ทฤษฎีและหลักการที่เกี่ยวข้อง

2.1 ระบบปฏิบัติการลินุกซ์ (Linux Operating System)

2.1.1 ลินุกซ์คืออะไร

ลินุกซ์ถือกำเนิดขึ้นโดยลินุส โตร์วัลดส์ (Linus Torvalds) โดยลินุสเห็นว่าระบบมินิกซ์ ที่เป็นระบบยูนิกซ์บนพีซีในขณะนั้น ยังมีความสามารถไม่เพียงพอแก่ความต้องการ จึงได้เริ่มพัฒนาระบบยูนิกซ์ของตัวเองขึ้นมา และแจกจ่ายให้ทดลองใช้เพื่อทดสอบหาจุดบกพร่อง ที่น่าสนใจก็คืองานต่างๆเหล่านี้ ผู้คนทั้งหมดที่ทำงาน ต่างก็พัฒนาโดยไม่คิดค่าตอบแทน และทำงานผ่านอินเทอร์เน็ตทั้งหมด

ลินุกซ์ (Linux) เป็นระบบปฏิบัติการแบบ 32 บิต ที่ได้ถูกพัฒนาขึ้นมาเป็นระบบปฏิบัติการที่ทำงานเสมือนระบบปฏิบัติการยูนิกซ์ (Unix Clone) บนเครื่องพีซี ซึ่งระบบปฏิบัติการยูนิกซ์ส่วนใหญ่ นั้นจะถูกใช้ในเครื่องเซิร์ฟเวอร์ขนาดใหญ่และมีราคาสูง ทำให้ได้เริ่มมีการพัฒนาให้ใช้ได้กับเครื่องพีซีธรรมดาที่มีราคาถูกกว่า โดยใช้แพลตฟอร์ม X86 เป็นแพลตฟอร์มเริ่มแรกก่อน จนกระทั่งได้มีการพัฒนาให้สามารถใช้งานลินุกซ์กับแพลตฟอร์มอื่นๆได้หลากหลายมากยิ่งขึ้น เช่น ARM, StrongARM, Alpha, Motorola Dragonball เป็นต้น โดยผู้ใช้สามารถที่จะนำเอาลินุกซ์ไปแจกจ่ายและใช้งานได้ฟรี เนื่องจากเป็นระบบปฏิบัติการที่ถูกพัฒนาขึ้นจากความร่วมมือของอาสาสมัครทั่วโลกซึ่งอาศัยการติดต่อกันผ่านทางอินเทอร์เน็ต และมีการวางแผนงานในการพัฒนาต่อเนื่องไปในระยะยาว ที่น่าสนใจคือ การร่วมมือในการพัฒนาของผู้พัฒนาทั้งหมดนี้ต่างก็ทำงานโดยความสมัครใจและไม่คิดค่าตอบแทนใดๆทั้งสิ้น จึงทำให้มั่นใจได้ว่าลินุกซ์นั้นเป็นระบบปฏิบัติการที่มีอนาคตไกล และยังคงมีการพัฒนาต่อไปเรื่อยๆอย่างไม่หยุดยั้ง

ในปัจจุบันนี้ ลินุกซ์นั้นมีความเข้ากันได้กับมาตรฐาน POSIX ซึ่งเป็นมาตรฐานอินเทอร์เฟซที่มีในระบบยูนิกซ์เป็นส่วนใหญ่ เพื่อทำให้สามารถที่จะนำโปรแกรมที่พัฒนาจากระบบยูนิกซ์อื่นๆ ที่ใช้มาตรฐานนี้ไปทำงานกับระบบอื่นๆที่รองรับมาตรฐานนี้ได้โดยไม่ต้องแก้ไขโปรแกรม ซึ่งทำให้ลินุกซ์สามารถใช้โปรแกรมที่ได้มีการพัฒนาไว้แล้วจากระบบยูนิกซ์อื่นๆมาใช้งานได้ทันที

2.1.2 ข้อดีของลินุกซ์

- 1). เนื่องจากเป็นระบบปฏิบัติการฟรี ไม่มีการเสียค่าใช้จ่ายในการซื้อ และเหมาะสมกับการใช้งานกับระบบฝังตัวใดๆ เนื่องจากสามารถควบคุมขนาดของระบบได้ และยังสามารถทำการแก้ไขข้อผิดพลาดและตรวจสอบความผิดพลาดได้ง่าย
- 2). เนื่องจากลินุกซ์เป็นที่แพร่หลาย ผู้ใช้ต่างๆได้แก้ไขให้สามารถทำงานได้บนหน่วยประมวลผลกลาง (CPU) ได้หลากหลาย นอกจากนี้ยังมีผู้พัฒนาแอปพลิเคชันประยุกต์ออกมาอย่างมากมาย
- 3). มีประสิทธิภาพและคุณภาพสูง เพราะด้วยว่าลินุกซ์เป็นระบบปฏิบัติการ 32 บิตเต็มรูปแบบซึ่งสามารถดึงเอาความสามารถของเครื่องคอมพิวเตอร์มาได้อย่างเต็มที่

4). มีคุณสมบัติของระบบยูนิกซ์เต็มรูปแบบเพราะมีพื้นฐานการพัฒนามาจากระบบยูนิกซ์ และเป็นระบบแบบผู้ใช้หลายคน (Multiuser) นอกจากนี้ลินุกซ์ยังมีระบบปฏิบัติการกราฟิกที่เรียกกันว่า X Windows ซึ่งเป็นมาตรฐานของระบบยูนิกซ์ทั่วไป และสามารถใช้ Windows Manager ได้หลากหลายแล้วแต่ความต้องการ นอกจากนี้ลินุกซ์ยังสนับสนุนโพรโทคอล TCP/IP, SLIP, PPP, UUCP และอื่นๆ

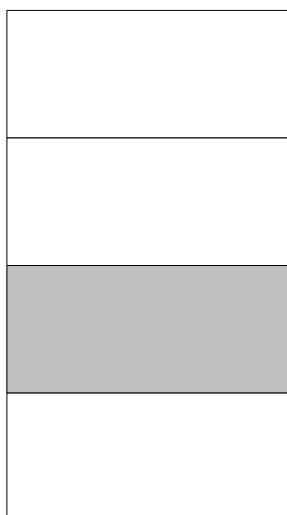
5). สามารถหาข้อมูลได้ง่าย เนื่องจากลินุกซ์เองนั้นเป็นระบบแบบเปิด (Open Source) และหากมีข้อสงสัยก็สามารถหาคำตอบจากอินเทอร์เน็ตได้ เนื่องจากว่ามีผู้พัฒนาต่างๆทั่วโลกให้การสนับสนุนซึ่งกันและกัน

2.1.3 ลินุกซ์เคอร์เนล (Linux Kernel)

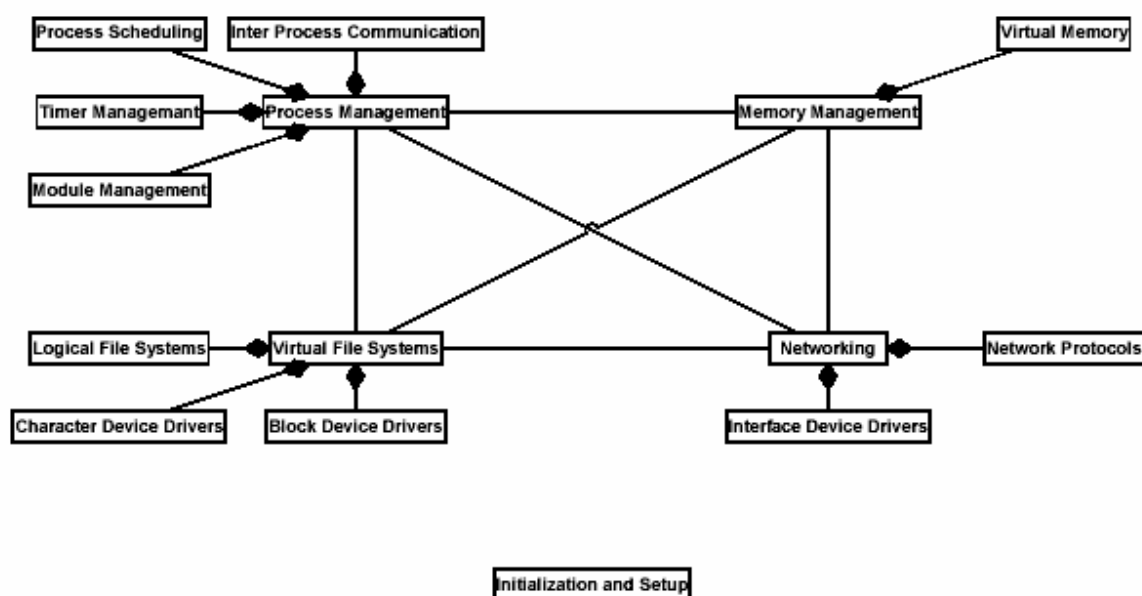
ลักษณะที่ต้องพิจารณาในการออกแบบลินุกซ์เคอร์เนล ได้แก่ ความชัดเจน (Clarity), ความเข้ากันได้ (Compatibility), สะดวกในการโยกย้าย (Portability), ทนทาน (Robustness), ปลอดภัย (Security), และเร็ว (Speed) ซึ่งสามารถแยกพิจารณาที่ละส่วนดังนี้

1. Clarity คือ ลินุกซ์เคอร์เนลที่ออกแบบขึ้นมาต้องมีความชัดเจน สามารถทำความเข้าใจได้ง่าย ทั้งส่วนของเรื่องความเร็วและความทนทาน โดยความชัดเจนในเรื่องความทนทาน หมายความว่าสามารถพิสูจน์ความถูกต้องของข้อมูลได้ง่าย และหากเกิดความผิดพลาดขึ้น สามารถทำการดีบั๊กได้ง่าย ระบบเกิดความเสียหายได้ยาก แต่ในส่วนของความชัดเจนกับความเร็วจะตรงข้ามกัน ถ้าในส่วนตัวคอมไพล์โค้ด ความชัดเจนจะสำคัญกว่า แต่ในการออกแบบเคอร์เนล ควรเน้นไปที่ความเร็ว เพราะผู้พัฒนาสามารถทำการอธิบายเกี่ยวกับความชัดเจนได้อยู่แล้ว
2. Compatibility ลินุกซ์เคอร์เนลสามารถนำไปใช้งานร่วมกับยูนิกซ์ได้ โดยสนับสนุนมาตรฐาน POSIX และยังใช้งานร่วมกับตัวอื่นๆ ได้แก่ สนับสนุนการรันไฟล์ .class ของจาวา สามารถทำการรันดอส (DOS) ผ่าน DOSEMU อิมูเลเตอร์ ทำการแชร์ไฟล์และเครื่องพิมพ์ได้เหมือนกับวินโดวส์โดยใช้ SAMBA สามารถใช้งานไคร์ฟจากระบบปฏิบัติการอื่นๆได้ รวมทั้งสนับสนุนระบบเน็ตเวิร์คด้วยโพรโทคอล TCP/IP
3. Protability ลินุกซ์เคอร์เนลสามารถทำงานร่วมกับฮาร์ดแวร์ โดยสามารถรันลินุกซ์เคอร์เนลบนฮาร์ดแวร์หลายแพลตฟอร์มได้ เมแรกทำการพัฒnalินุกซ์บนสถาปัตยกรรม X86 แต่ปัจจุบันทำการพัฒnalินุกซ์เคอร์เนลที่สามารถใช้งานบนสถาปัตยกรรมหลายแพลตฟอร์มได้แก่ อัลฟา, อาร์ม, โมโตโรล่า, MIPS, พาวเวอร์พีซี, SPARC และ SPARC-64 เป็นต้น ยิ่งไปกว่านั้น สามารถรันลินุกซ์บน Amigas, Mac, เวิร์กสเตชัน และ SGI, NeXT และอื่นๆอีกมากมายการที่ลินุกซ์สามารถทำงานบนหลายแพลตฟอร์มเนื่องจากว่า โค้ดของลินุกซ์เคอร์เนล สามารถนำไปใช้งานได้หลากหลาย
4. Robustness และ Security ลินุกซ์เคอร์เนลที่พัฒนาขึ้นต้องมีความทนทานและปลอดภัย ควรจะมีการป้องกันระบบจากระบบอื่น โดยเฉพาะทำการป้องกันโปรเซสที่ทำงานจากส่วนอื่นๆ สามารถทำการดีบั๊ก และแก้ไขข้อผิดพลาดได้ง่าย

5. Speed ในการออกแบบลินุกซ์เคอร์เนล ส่วนที่ต้องพิจารณาลดับแรก ซึ่งมีความสำคัญมาก ได้แก่ ความเร็ว ตามมาด้วยความทนทาน ความปลอดภัย และความเข้ากันได้ เพราะสามารถตรวจสอบประสิทธิภาพการทำงานของระบบได้จากความเร็วในการทำงาน การรันงานต่างๆ ซึ่งบางครั้งสามารถจับเวลาในการรันโปรเซสด้วยตัวเองได้



รูปที่ 2-1 แสดงชั้นการทำงานของลินุกซ์เคอร์เนล



รูปที่ 2-2 รายละเอียดเพิ่มเติมของลินุกซ์เคอร์เนล

โดยสถาปัตยกรรมหลักของลินุกซ์เคอร์เนล จะประกอบด้วย 5 ส่วนหลัก ดังนี้

1. Process Management

ทำหน้าที่สร้าง และทำลายโปรเซส และทำการรองรับโปรเซสที่ติดต่อกับภายนอก และโปรเซสที่ติดต่อกันเองภายใน รวมถึงจัดการลำดับการทำงานของโปรเซส

2. Memory Management

ทำหน้าที่จัดสรรพื้นที่ว่างในหน่วยความจำให้กับโปรเซสให้มีประสิทธิภาพ รวมทั้งการจัดการคืนพื้นที่ว่างในโปรเซสที่ไม่ใช้แล้วโดยผ่านชุดฟังก์ชันของ call

3. Filesystems

เนื่องจากระบบปฏิบัติการลินุกซ์อยู่บนพื้นฐานของแนวคิดระบบไฟล์ (Filesystems) โดยจะมองส่วนต่างๆเป็นไฟล์ และจะทำการสร้างโครงสร้างของระบบไฟล์บนฮาร์ดแวร์ ซึ่งจะจัดการกับข้อมูลบนสื่อที่แตกต่างกัน

4. Networking

เนื่องจากว่าโอเปอเรชันของเน็ตเวิร์คส่วนใหญ่แล้วไม่ได้ระบุในโปรเซส แพ็กเกจที่เข้ามาเป็นเหตุการณ์แบบอะซิงโครนัส ดังนั้นต้องมีส่วนที่ทำหน้าที่ในการจัดการการติดต่อสื่อสารกันในเครือข่าย โดยผ่านทางเน็ตเวิร์คอินเตอร์เฟซ (Network Interface) ซึ่งจะทำให้โปรเซสต่างๆสามารถที่จะร้องขอการให้บริการทางด้านระบบเครือข่ายได้ถูกต้องโดยง่าย

5. Device Control

ทำหน้าที่ในการควบคุมอุปกรณ์ให้ทำงานได้อย่างถูกต้องตามต้องการของโปรเซสโดยไม่ต้องเขียนติดต่อกับอุปกรณ์โดยตรง ผ่านทางไดรเวอร์ และเคอร์เนลเองต้องมีส่วนของดีไวซ์ไดรเวอร์สำหรับอุปกรณ์ที่เชื่อมต่อกับระบบ เช่นไดรเวอร์ของฮาร์ดดิสก์ คีย์บอร์ด จอภาพ เป็นต้น

2.1.4 ลักษณะเด่นของลินุกซ์

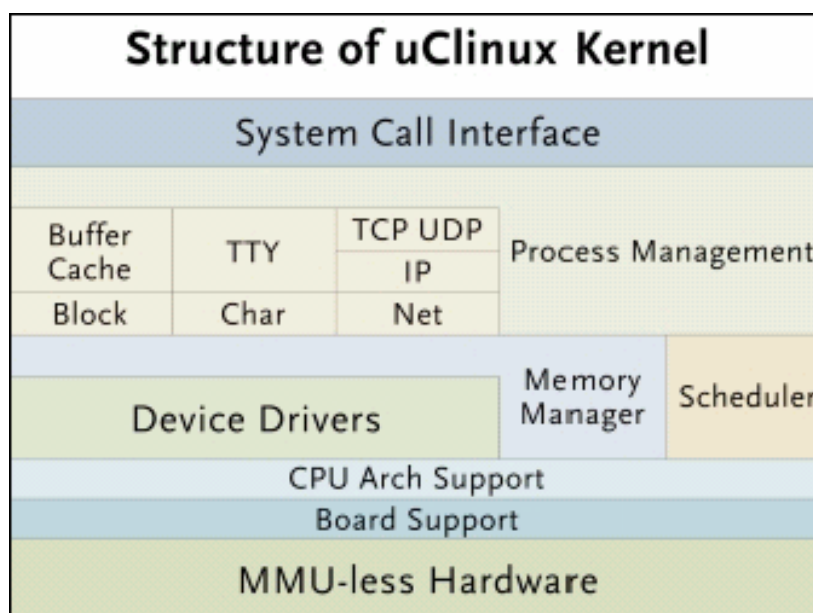
1. เป็นระบบปฏิบัติการแบบ 32 บิตที่เป็นยูนิกซ์โคลนสำหรับเครื่องพีซีทั่วไป และไม่เสียค่าใช้จ่ายในการซื้อ
2. สนับสนุนการใช้งานที่หลากหลายงาน และหลายผู้ใช้งาน (Multitasking and Multiuser)
3. ปกป้องหน่วยความจำ (Memory Protection)
4. หน่วยความจำเป็นแบบเสมือน (Virtual Memory)
5. มีระบบ X-Windows ซึ่งเป็นระบบการติดต่อผู้ใช้แบบกราฟฟิกที่ไม่ขึ้นกับระบบปฏิบัติการหรือฮาร์ดแวร์ใดๆ
6. มีมาตรฐาน TCP/IP ที่เป็นมาตรฐานที่ใช้บนอินเทอร์เน็ต
7. สามารถเข้ากันได้กับมาตรฐาน POSIX ซึ่งเป็นมาตรฐานอินเตอร์เฟซที่ระบบยูนิกซ์ส่วนใหญ่ต้องมี และรูปแบบบางส่วนคล้ายระบบปฏิบัติการยูนิกซ์ของ Berkeley และ System V
8. มีความเร็วและประสิทธิภาพสูงมาก

2.2 uCLinux

เนื่องจากในปัจจุบัน หน่วยประมวลผลหรือ CPU มักจะไม่มีส่วนของการจัดการหน่วยความจำ หรือ MMU (Memory Management Unit) แต่เนื่องจากมาตรฐานของลินุกซ์เคอร์เนลจะต้องการส่วนจัดการหน่วยความจำ ดังนั้น เพื่อให้ลินุกซ์สามารถทำงานได้บนอุปกรณ์ที่ไม่สนับสนุนส่วนนี้ จึงเกิดลินุกซ์ขึ้นมาเพื่ออุปกรณ์แบบฝังตัว หรือไมโครคอนโทรลเลอร์ลินุกซ์ (uClinux) โดยทำการเอาส่วนที่ต้องการจัดการหน่วยความจำออกจากลินุกซ์ แล้วแทนที่ด้วยโมเดลหน่วยความจำแบบแฟลช ซึ่งสถาปัตยกรรมใหม่ที่เกิดขึ้นยังคงความสามารถของเคอร์เนลและเปิดเผยโค้ดอยู่

uClinux พัฒนาโดย Dr. Jeff Dionne และ Kenneth Albanowski เกิดมาจากลินุกซ์เคอร์เนลเวอร์ชัน 2.0 ซึ่งเป็นฉบับย่อแต่ยังคงข้อดีไว้ ได้แก่ ความมีเสถียรภาพ ความสามารถในการติดต่อกับเน็ตเวิร์ค สนับสนุนระบบไฟล์ มาตรฐาน API รวมทั้งขนาดโค้ด จะมีขนาดเล็กกว่าลินุกซ์ธรรมดา โดยมีวัตถุประสงค์เพื่อใช้งานกับไมโครคอนโทรลเลอร์ที่ไม่มีส่วนจัดการหน่วยความจำ และมุ่งเน้นที่ระบบฝังตัว เพราะโปรเซสเซอร์ที่ไม่มีส่วนจัดการหน่วยความจำนั้นราคาถูก จึงเหมาะสมกับการนำไปใช้ในระบบฝังตัว

ลักษณะการทำงานจะเป็นแบบหลายงานพร้อมกัน และสามารถนำเอาแอปพลิเคชันต่าง ๆ มาทำงานบนตัวมันเองได้โดยไม่จำเป็นต้องเป็นแบบหลายงานพร้อมกัน



รูปที่ 2-3 โครงสร้างของระบบปฏิบัติการ uClinux

2.2.1 คุณลักษณะเด่นของ uClinux

1. สนับสนุนมาตรฐาน API
2. เคอร์เนลมีขนาดเล็กกว่า 512 กิโลไบต์
3. ขนาดของเคอร์เนล รวมกับทูลต่างๆ มีขนาดเล็กกว่า 900 กิโลไบต์

4. มี TCP/IP Stack ทำให้สามารถใช้งานผ่านอินเทอร์เน็ตได้
5. สนับสนุนโปรโตคอลเชื่อมต่อกับเน็ตเวิร์ค
6. สนับสนุนระบบไฟล์ ประกอบด้วย NFS, ext2, ROMfs, JFFS (Journaling FLASH File System), MS-DOS และ FAT 16/32
7. มีการบริหารแบบ Preemptive
8. มีความสามารถในการทำงานหลายงานพร้อมกัน
9. สนับสนุนมาตรฐาน POSIX4 คือเพิ่มความสามารถทางด้านเรียลไทม์เข้าไป
10. สามารถทำงานแอปพลิเคชันได้ทั้งแบบ Single และ Multitread
11. ทำงานภายใต้โปรเซสเซอร์ได้หลายตระกูล ซึ่งในปัจจุบันนี้ มีหน่วยประมวลผลที่สามารถใช้งานได้มากมาย ไม่ว่าจะเป็น Motorola DragonBall, Intel Strong ARM และอื่นๆ

2.2.2 ข้อจำกัดของ uClinux

เนื่องจากว่า uClinux ใช้กับโปรเซสเซอร์ที่ไม่มีส่วนจัดการหน่วยความจำ ทำให้มีข้อจำกัดดังนี้

1. ไม่สามารถขยายขนาดของสแตคได้ และไม่มี brk() ซึ่งทำหน้าที่ในการขยายขนาดหน่วยความจำตามที่ร้องขอ โดยใช้ mmap() แทนเพื่อจองหน่วยความจำ
2. ไม่สามารถใช้งาน fork() ได้ จึงมี vfork() มาทำงานแทน ซึ่งไม่ได้หมายความว่าไม่สามารถทำงานหลายอย่างพร้อมกันได้ แต่หมายถึงโปรเซสแม่ จะทำการคอยจนกว่าโปรเซสลูกทำ exec() หรือ exit() ซึ่งยังคงใช้งานพร้อมกันได้อย่างเต็มที่
3. ไม่มีการป้องกันหน่วยความจำ ทำให้สามารถเกิดความเสียหายของโปรแกรมต่างๆ รวมทั้งเคอร์เนล
4. สถาปัตยกรรมที่ใช้มีขนาดของโค้ดที่ต่างกัน ขึ้นอยู่กับรูปแบบที่นำไปใช้งาน

2.2.3 ข้อดีของ uClinux

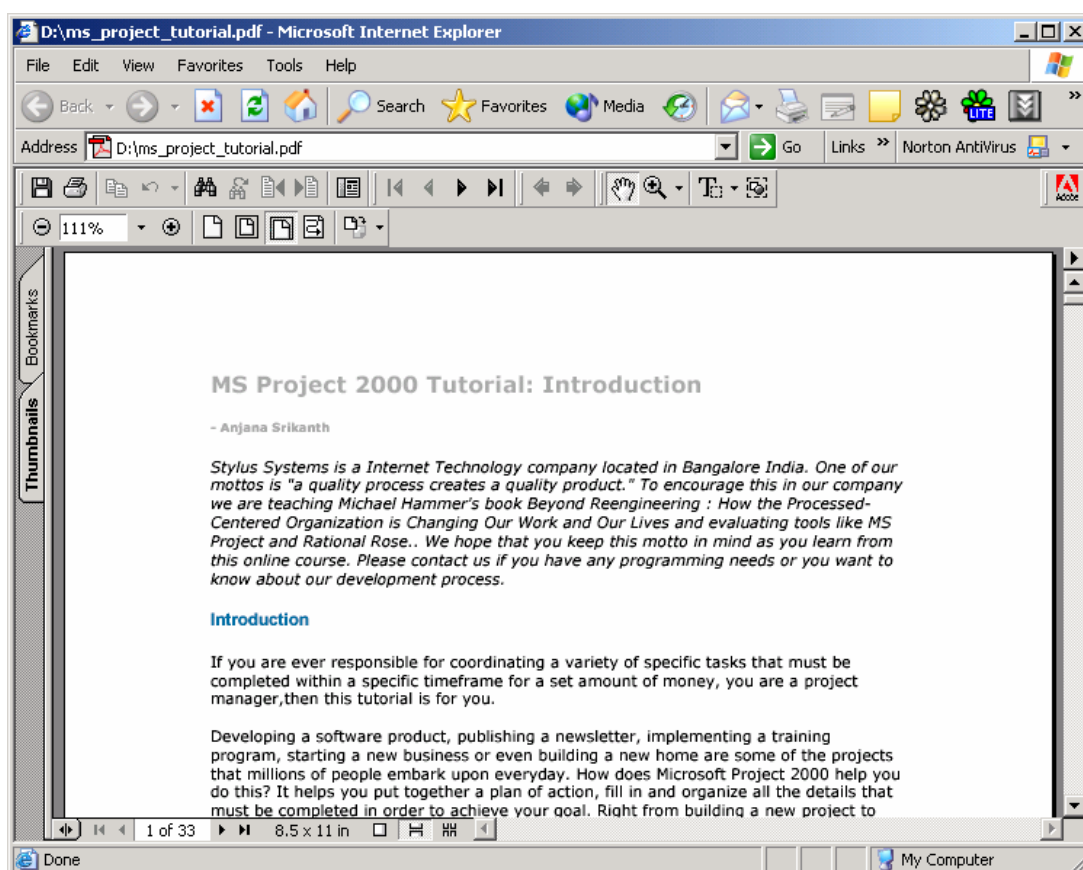
1. ขนาดเล็ก
2. เปิดเผยโค้ด
3. เหมาะในการนำไปใช้กับโปรเซสเซอร์ที่ไม่มี MMU โดยเฉพาะระบบฝังตัว
4. เหมาะกับระบบราคาถูก และใช้พลังงานต่ำ โดยเฉพาะระบบฝังตัว

2.2.4 สรุป

เลือกใช้ uClinux เพราะว่ามีขนาดเล็ก เหมาะในการนำไปใช้กับโปรเซสเซอร์ที่ไม่มี MMU ใช้พลังงานต่ำเหมาะกับระบบฝังตัว โดยอาจจะเลือก Pocket Linux ซึ่งอิงกับรูปแบบ XML และ มีขนาดใหญ่ โดยอาจจะมีตัวที่ดีกว่าในอนาคต

2.3 ปลั๊กอิน (Plug-in)

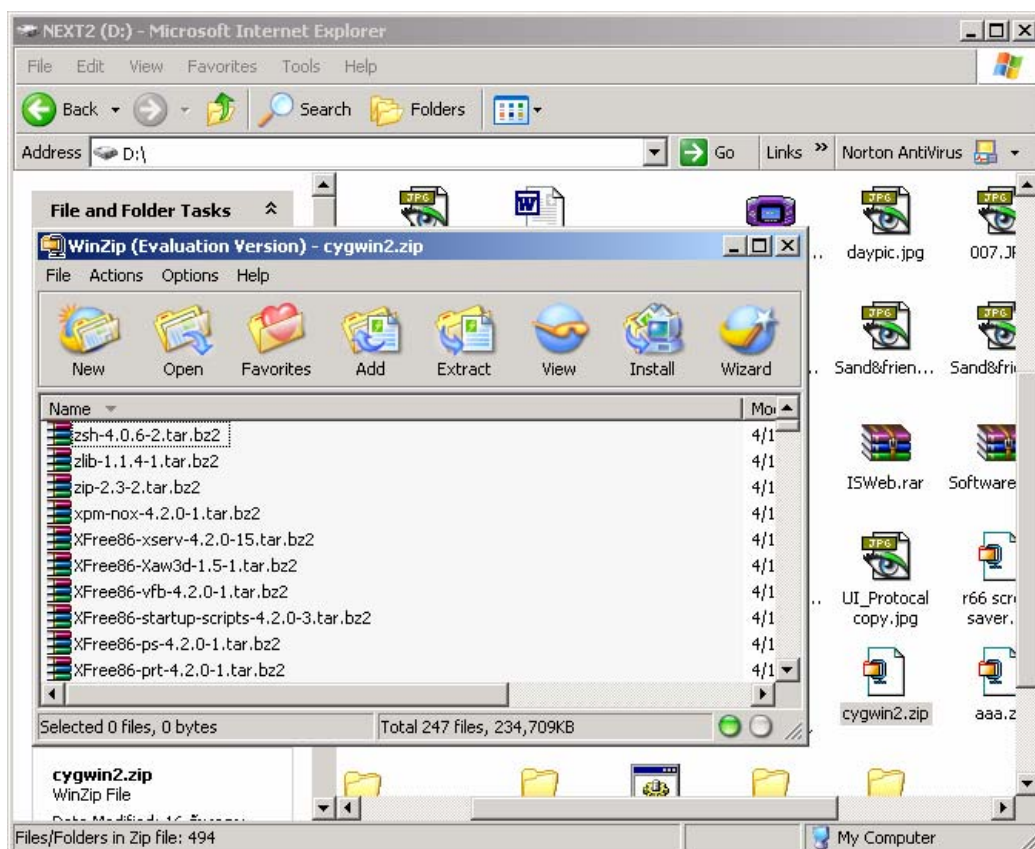
ปลั๊กอิน คือ โมดูลโค้ดที่แยกต่างหากออกมา ซึ่งทำงานเป็นส่วนหนึ่งของแอปพลิเคชันใด แอปพลิเคชันหนึ่งที่เรียกขึ้นมา โดยที่ปลั๊กอินสามารถเป็นได้หลายแบบ ไม่ว่าจะเป็นแอปพลิเคชันที่อยู่ภายนอก หรือว่าเป็นฟังก์ชันเพิ่มเติมเข้าไป เพื่อเพิ่มสามารถของแอปพลิเคชันเดิม ซึ่งจะมีปลั๊กอินเอพีไอ (Plug-in API) เป็นส่วนที่ติดต่อระหว่างแอปพลิเคชันที่เรียก กับปลั๊กอินที่ถูกเรียก ซึ่งทำให้ลดขอบเขตความสามารถของแอปพลิเคชันเดิมลงไป



รูปที่ 2-4 แสดงปลั๊กอินของโปรแกรมโครเบต รีดเดอร์ (Acrobat Reader) ซึ่งทำงานบนโปรแกรมอินเทอร์เน็ต เอ็กซ์พลอเรอร์ (Internet Explorer)

2.3.1 ข้อแตกต่างระหว่างปลั๊กอิน และแอปพลิเคชันที่เปิดขึ้นมาแยกต่างหาก

เมื่อแอปพลิเคชันหลักต้องการเปิดไฟล์ใดไฟล์หนึ่ง แอปพลิเคชันนั้นจะทำการเช็คชนิดของไฟล์ (MIME Type) ที่มีความสัมพันธ์กับไฟล์ที่จะเปิด จากนั้นก็จะค้นหาปลั๊กอินที่เหมาะสม ถ้าเกิดเรียกปลั๊กอินขึ้นมา ตัวปลั๊กอินจะถูกเรียกขึ้นมาแสดงผล หรือทำงานบนเนื้อที่ที่แอปพลิเคชันหลักจัดเตรียมไว้ให้ แต่ถ้าเกิดไม่เจอปลั๊กอินที่เหมาะสม แอปพลิเคชันหลักก็จะเรียกแอปพลิเคชันภายนอกขึ้นมาทำงาน ซึ่งจะเปิดขึ้นมาทำงานบนเนื้อที่ใหม่ ไม่ได้อยู่บนเนื้อที่ที่แอปพลิเคชันหลักจัดเตรียมไว้ให้



รูปที่ 2-5 แสดงโปรแกรมวินซิป (Winzip) ที่ต้องเปิดขึ้นมาต่างหาก
เพราะว่าไม่ได้เป็นปลั๊กอินของโปรแกรมอินเทอร์เน็ตเอ็กโพรเลอร์

2.3.2 ปลั๊กอินทำงานอย่างไร

เมื่อแอปพลิเคชันหลักเปิดขึ้นมา มันจะเช็คปลั๊กอินโมดูลจากโฟลเดอร์ที่ได้กำหนดไว้ ซึ่งปลั๊กอินหนึ่ง อาจจะมีมีความเกี่ยวเนื่องกับชนิดของไฟล์เพียงชนิดเดียว หรือมากกว่านั้น จากนั้น แอปพลิเคชันหลักจะพิจารณาว่าปลั๊กอินนั้นเหมาะสมกับชนิดไฟล์แบบใด แล้ววีจิเตอร์โลบรารีปลั๊กอินสำหรับชนิดไฟล์ที่เหมาะสม เมื่อผู้ใช้ เปิดไฟล์ขึ้นมา ซึ่งจำเป็นต้องใช้ปลั๊กอิน ก็จะมีลำดับการทำงานดังนี้

- เช็คปลั๊กอินที่เหมาะสมกับชนิดของไฟล์
- โหลดปลั๊กอินเข้าไปยังหน่วยความจำ
- เริ่มจัดเตรียมค่าต่างๆ ที่ปลั๊กอินต้องการ
- เริ่มให้ปลั๊กอินทำงาน

เมื่อผู้ใช้เปลี่ยนการทำงานไป หรือปิดแอปพลิเคชันลงไป ปลั๊กอินที่ทำงานก็จะถูกปิดการทำงานลง ไป แต่ค่าต่างๆ ที่ปลั๊กอินต้องการ ยังคงอยู่ จนทุกปลั๊กอินถูกปิดการทำงานไปแล้ว ค่าต่างๆ ของปลั๊กอินที่ยังไม่ถูกนำออกจากหน่วยความจำก็ถูกนำออกไป

2.3.3 ชนิดของไฟล์ปลั๊กอิน

ชนิดของไฟล์ปลั๊กอิน จะขึ้นอยู่กับแพลตฟอร์มต่างๆ ซึ่งมีดังนี้

- ไมโครซอฟท์ วินโดวส์ จะเป็นไฟล์ประเภทไดนามิกลิงก์ไลบรารี (.DLL)
- ยูนิกซ์ จะเป็นไฟล์ประเภทแชร์ออฟเจ็ค (.SO หรือ .DSO)
- แมค โอเอส จะเป็นไฟล์ประเภทเพาเวอร์พีซี แชร์ไลบรารี หรือ 68เค โค้ด รีซอส (PowerPC Shared Library or 68K code resources)